

XANATZ

A Comprehensive Freelancer Marketplace & RFP Bidding Platform

Document Type:	System Documentation & Architecture Manual
Version:	1.0.0 (Production Release)
Date of Compilation:	July 13, 2026
Author/Developer:	Software Engineering Project Team
Language & Stack:	React v18 + Django REST Framework + MySQL

DOCUMENTATION INDEX

1. Introduction	• Project Overview • Purpose of the Project • Scope & Objectives	Page 3
2. Requirements Analysis	• Functional Requirements • Non-Functional Requirements • URS	Page 3
3. System Requirements	• Hardware & Software Requirements • Browsers & Dev Tools	Page 4
4. Technology Stack	• Frontend & Backend Technologies • DBMS • Libraries & APIs	Page 4
5. System & Database Design	• System Architecture • UML Diagrams • DFDs • Data Dictionary	Page 5
6. Module Design	• User Auth • Dashboard • Functional & Admin Modules	Page 8
7. Functional Features	• Role-Based Dashboards • Security Details	Page 9
8. RESTful API Specification	• API Architecture • Endpoints • Formats • Error Handling	Page 9
9. User Interface Design	• Home Page • Auth Screens • Dashboards • Responsive UI	Page 11
10. System Testing	• Test Strategy • Core Test Cases • Test Results Summary	Page 11
11. Security Implementation	• JWT Authentication • Hashing • CSRF & Input Sanitization	Page 12
12. Future Enhancements	• Payment Gateways • Video Modules • Recommendation System	Page 13
13. Conclusion	• Final Project Summary & Delivery Metrics	Page 13
14. References	• Official Framework Documents & References	Page 13
15. Appendices	• Database SQL Scripts • API FAQ • User Manual Guide	Page 14

1. Introduction

1.1 Project Overview

Xanatz is an advanced, production-grade digital marketplace designed to bridge the gaps in modern gig hiring. It acts as a secure platform connecting freelance specialists with corporate clients. By integrating standard job portal tools with a full Requests for Proposals (RFP) bidding engine, it provides a unified hub for job postings, contract bidding, portfolio hosting, and real-time correspondence.

1.2 Purpose of the Project

Traditional freelancer platforms are often burdened by high fees, opaque rating metrics, and disconnected workspaces. The purpose of Xanatz is to create a seamless, end-to-end recruitment cycle with secure OTP authentication, interactive client-freelancer dashboard reporting, direct chat messaging, and structured feedback mechanisms that enhance transparency and professional credibility.

1.3 Project Scope

The application scope encapsulates secure authentication, freelancer profile curation, interactive portfolio listings, company page management, subscription plan controls, job opening applications, and request for proposal submissions. Additionally, it supports direct instant messaging and dynamic feedback models.

1.4 Project Objectives

- Design and implement a modern, high-fidelity responsive frontend using React and Chakra UI.
- Develop a robust, secure, and authenticated backend using Django REST Framework.
- Establish a relational data layer with key constraints, indices, and data dictionaries.
- Enable seamless asynchronous file uploading for applicant resumes and project proposals.
- Secure all communications with JWT tokenization, CORS, and SQL sanitization layers.

2. Requirements Analysis

2.1 Functional Requirements

- **Authentication:** User registration, OTP code delivery via SMTP, and JWT-secured login.
- **Profile Curation:** Edit headline, skills, work experiences, academic histories, and portfolios.
- **Company Panel:** Register corporate profiles, purchase posting credits, and manage member roles.
- **Job Postings:** Create, edit, and filter openings. Apply using PDF/DOCX resume attachments.
- **RFP Engine:** Submit project specifications, budget, and deadlines. Bidders upload detailed proposal files.
- **Messaging:** Native asynchronous direct messaging interface between verified users.
- **Review Engine:** Dual-sided review module allowing companies to rate freelancers and vice versa.

2.2 Non-Functional Requirements

- **Security:** Implement JWT token validation, HTTPOnly cookies, CORS filters, and password hashing.
- **Performance:** Maintain page response latencies below 2.0s under standard workloads.
- **Scalability:** Support concurrent user access via stateless APIs and database index optimization.

- **Usability:** Deliver cross-platform support with responsive web views and intuitive layouts.

2.3 User Requirements Specification (URS)

User roles are categorized into Freelancer (bidders/applicants), Company Admins (hirers/RFP owners), and System Admins. The system must adapt views dynamically: Company profiles view applicant lists and subscription credits, while Freelancers view biddable contracts, job openings, and applications history.

3. System Requirements

3.1 Hardware Requirements

- **Development System:** Intel Core i5 / AMD Ryzen 5, 8GB RAM, 20GB free storage space.
- **Deployment Server:** VPS with 1vCPU, 2GB RAM, 30GB SSD Storage (standard cloud hosting).

3.2 Software Requirements

- **Operating System:** Linux (Ubuntu 20.04 LTS or newer), macOS, or Windows 10/11.
- **Runtimes & Frameworks:** Python 3.10+, Node.js v18+.
- **Database Management:** MySQL Community Server 8.0 or SQLite (development).

3.3 Supported Browsers

- Google Chrome (v100+), Mozilla Firefox (v98+), Apple Safari (v15+), and Microsoft Edge.

3.4 Development Environment and Tools

- VS Code, Git for version control, Postman for API testing, npm for packages, and Python virtualenv.

4. Technology Stack

4.1 Frontend Technologies

- **React (v18):** Multi-component structure with client-side routing via React Router DOM.
- **Chakra UI (v3):** Modern CSS tokens, responsive flexbox/grid containers, and custom theme layouts.
- **Framer Motion:** Controls custom page animations, slide transitions, and micro-interactions.
- **Axios:** Lightweight HTTP client for API call execution with request interceptors for token attachment.

4.2 Backend Technologies

- **Django Framework:** Relational ORM database interaction, middleware filtering, and core security modules.
- **Django REST Framework (DRF):** Exposes serializable RESTful API endpoints for frontend consumption.
- **SimpleJWT:** Secures communication using transient Access and Refresh JSON Web Tokens.

4.3 Database Management System (DBMS)

- **MySQL 8.0:** Handles ACID-compliant structured transactions, key constraints, and relational indices.

4.4 Frameworks and Libraries

- PyMySQL / mysqlclient drivers, python-dotenv config, reportlab PDF engine, and Pillow image manipulator.

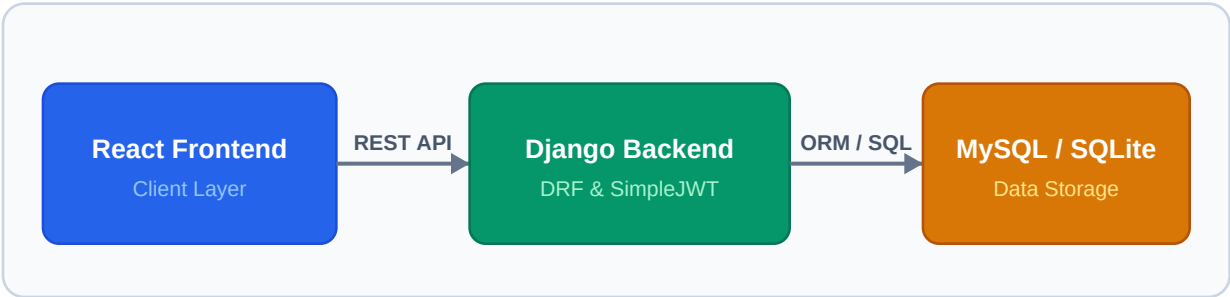
4.5 Third-Party Integrations / APIs

- SMTP services (SendGrid/Gmail) for email delivery, Google Auth APIs (optional), and Lucide Icons.

5. System Design and Database Design

5.1 System Architecture

Xanatz utilizes a modern Client-Server Architecture. The frontend layer runs in the client browser, handling state and UI display. The backend application runs statelessly, verifying requests using JWT tokens and querying the MySQL database to retrieve or update records.



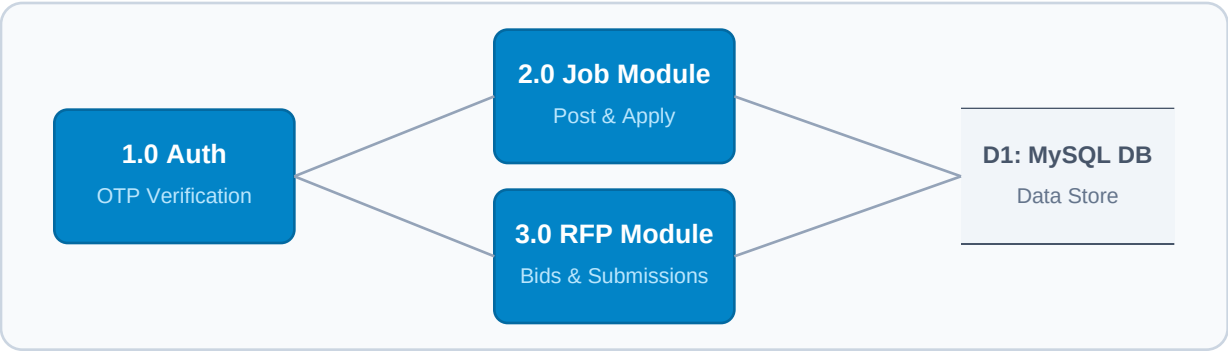
5.2 UML Design

The use case diagram illustrates user interaction. Freelancers interact with the profiles, job applications, and RFP interest systems. Companies interact with job openings, subscriptions, and RFP listings. All authenticated sessions pass through the OTP security verification process.



5.3 Data Flow Modeling

Data Flow Diagram Level 1 traces how files (like resumes and proposals) and messages traverse process bubbles to land inside ACID-compliant relational storage files.



5.4 Database Design

The database models are designed to ensure strict relational integrity and prevent data anomalies. Every table utilizes primary keys and foreign key constraints with cascade deletions where appropriate. The application schema consists of 19 relational tables mapped via Django ORM to MySQL 8.0.

System Relational Models and Table Mapping

Model Class Name	Database Table Name	Purpose & System Role
User	accounts_user	Core authentication model (email-based credentials and system flags).
Profile	accounts_profile	Freelancer profile holding bio, rates, location, and skills.
PortfolioProject	accounts_portfoliopproject	Portfolio projects showcasing freelancer work (links and images).
Experience	accounts_experience	Freelancer work timeline entries (companies, roles, and dates).
Education	accounts_education	Freelancer academic timeline entries (schools, degrees, fields).
OTP	accounts_otp	One-time password tokens generated for secure 2FA/login verification.
PrivacyPolicy	accounts_privacypolicy	Terms and privacy policy markdown documents storage.
Company	accounts_company	Employer organizations hosting job postings and RFP projects.
CompanyMember	accounts_companymember	Role access levels (Admin/User) of users inside companies.
JobOpening	accounts_jobopening	Job postings published by company accounts.
JobApplication	accounts_jobapplication	Logs, resume attachments, and status of applications to job openings.
RFP	accounts_rfp	Requests For Proposals issued by companies seeking vendor services.
RFPInterest	accounts_rfpinterest	Bids, proposals, and pricing documents submitted by agencies/freelancers.
JobPostPlan	accounts_jobpostplan	Pricing and credit parameters for basic, standard, and premium plans.
CompanySubscription	accounts_companysubscription	Active plan subscriptions and remaining job post credits.
Notification	accounts_notification	System logs and alerts pushed to users' notification centers.
Message	accounts_message	Direct messaging logs exchanged between workspace participants.
CompanyReview	accounts_companyreview	Ratings and reviews left by freelancers for employer accounts.
FreelancerReview	accounts_freelancerreview	Ratings and reviews left by employers for freelancer accounts.

Database Data Dictionary

Table: accounts_user

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
password	CharField(128)	NO		
last_login	DateTimeField	YES		
is_superuser	BooleanField	NO		False
first_name	CharField(150)	YES		
last_name	CharField(150)	YES		
is_staff	BooleanField	NO		False
is_active	BooleanField	NO		True
date_joined	DateTimeField	NO		timezone.now
email	EmailField	NO	UNI	
phone_number	CharField(15)	YES		
accepted_privacy_policy	BooleanField	NO		False

Table: accounts_profile

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
user_id	OneToOneField	NO	FK	
public_id	UUIDField	NO	UNI	uuid.uuid4
headline	CharField(255)	YES		
about	TextField	YES		
location	CharField(255)	YES		
profile_picture	ImageField	YES		
cover_image	ImageField	YES		
website	URLField	YES		
skills	JSONField	YES		list
is_freelancer	BooleanField	NO		False
hourly_rate	DecimalField	YES		
freelancer_currency	CharField(10)	NO		'USD'
freelancer_availability	CharField(20)	NO		'available'

Table: accounts_portfolioproject

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
profile_id	ForeignKey	NO	FK	
title	CharField(255)	NO		
description	TextField	YES		
image	ImageField	YES		
project_url	URLField	YES		
technologies	JSONField	YES		list

Table: accounts_experience

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
profile_id	ForeignKey	NO	FK	
company	CharField(255)	NO		
title	CharField(255)	NO		
location	CharField(255)	YES		
start_date	DateField	NO		
end_date	DateField	YES		
current	BooleanField	NO		False
description	TextField	YES		
company_website	URLField	YES		

Table: accounts_education

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
profile_id	ForeignKey	NO	FK	
school	CharField(255)	NO		
degree	CharField(255)	YES		
field_of_study	CharField(255)	YES		
start_date	DateField	NO		
end_date	DateField	YES		
description	TextField	YES		

Table: accounts_otp

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
email	EmailField	NO		
otp	CharField(6)	NO		
created_at	DateTimeField	NO		
is_verified	BooleanField	NO		False

Table: accounts_privacypolicy

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
content	TextField	NO		
updated_at	DateTimeField	NO		
created_at	DateTimeField	NO		

Table: accounts_companymember

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
user_id	ForeignKey	NO	FK	
company_id	ForeignKey	NO	FK	
access_role	CharField(20)	NO		'user'
position	CharField(255)	YES		
joined_at	DateTimeField	NO		

Table: accounts_company

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
name	CharField(255)	NO		
public_id	UUIDField	NO	UNI	uuid.uuid4
tagline	CharField(255)	YES		
description	TextField	YES		
logo	ImageField	YES		
website	URLField	YES		
industry	CharField(50)	YES		
company_size	CharField(20)	YES		
location	CharField(255)	YES		
founded_year	PositiveIntegerField	YES		
linkedin_url	URLField	YES		
twitter_url	URLField	YES		
is_active	BooleanField	NO		True
creator_id	ForeignKey	YES	FK	
members	ManyToManyField	YES		
created_at	DateTimeField	NO		
updated_at	DateTimeField	NO		

Table: accounts_jobopening

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
company_id	ForeignKey	NO	FK	
title	CharField(255)	NO		
description	TextField	NO		
requirements	TextField	YES		
location	CharField(255)	YES		
job_type	CharField(50)	NO		'full_time'
salary_range	CharField(100)	YES		
is_active	BooleanField	NO		True
expires_at	DateTimeField	YES		
created_at	DateTimeField	NO		
updated_at	DateTimeField	NO		

Table: accounts_jobapplication

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
job_opening_id	ForeignKey	NO	FK	
applicant_id	ForeignKey	NO	FK	
full_name	CharField(255)	NO		
email	EmailField	NO		
cover_letter	TextField	YES		
resume	FileField	YES		
portfolio_url	URLField	YES		
key_skills	TextField	YES		
status	CharField(50)	NO		'applied'
created_at	DateTimeField	NO		
updated_at	DateTimeField	NO		

Table: accounts_rfp

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
company_id	ForeignKey	NO	FK	
title	CharField(255)	NO		
description	TextField	NO		
budget	CharField(100)	YES		
deadline	DateField	YES		
requirements	TextField	YES		
category	CharField(100)	YES		
sub_category	CharField(100)	YES		
is_active	BooleanField	NO		True
created_at	DateTimeField	NO		
updated_at	DateTimeField	NO		

Table: accounts_rfpinterest

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
rfp_id	ForeignKey	NO	FK	
user_id	ForeignKey	NO	FK	
company_name	CharField(255)	NO		
email	EmailField	NO		
phone_number	CharField(15)	YES		
proposal_summary	TextField	NO		
attached_file	FileField	YES		
status	CharField(50)	NO		'pending'
is_reviewed	BooleanField	NO		False
created_at	DateTimeField	NO		

Table: accounts_jobpostplan

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
name	CharField(50)	NO	UNI	
display_name	CharField(50)	NO		
price	DecimalField	NO		0
max_jobs	PositiveIntegerField	NO		
job_duration_days	PositiveIntegerField	NO		
description	TextField	YES		
features	JSONField	YES		list
is_active	BooleanField	NO		True
created_at	DateTimeField	NO		

Table: accounts_companysubscription

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
company_id	ForeignKey	NO	FK	
plan_id	ForeignKey	NO	FK	
jobs_used	PositiveIntegerField	NO		0
activated_at	DateTimeField	NO		
is_active	BooleanField	NO		True

Table: accounts_notification

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
recipient_id	ForeignKey	NO	FK	
sender_id	ForeignKey	YES	FK	
message	TextField	NO		
is_read	BooleanField	NO		False
created_at	DateTimeField	NO		
target_url	CharField(255)	YES		

Table: accounts_message

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
sender_id	ForeignKey	NO	FK	
recipient_id	ForeignKey	NO	FK	
content	TextField	NO		
is_read	BooleanField	NO		False
created_at	DateTimeField	NO		

Table: accounts_companyreview

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
reviewer_id	ForeignKey	NO	FK	
company_id	ForeignKey	YES	FK	
rfp_interest_id	ForeignKey	YES	FK	
company_name	CharField(255)	NO		
rating	PositiveIntegerField	NO		
review_text	TextField	NO		
is_flagged	BooleanField	NO		False
created_at	DateTimeField	NO		

Table: accounts_freelancerreview

Field Name	Type	Null	Key	Default
id	AutoField	NO	UNI	
reviewer_id	ForeignKey	NO	FK	
freelancer_id	ForeignKey	NO	FK	
rfp_interest_id	ForeignKey	YES	FK	
rating	PositiveIntegerField	NO		
review_text	TextField	NO		
is_flagged	BooleanField	NO		False
created_at	DateTimeField	NO		

6. Module Design

6.1 User Authentication and Authorization

Verifies logins via temporary 6-digit OTP codes. Upon validation, the server generates two JSON Web Tokens: a short-lived Access Token (15 mins) and a long-lived Refresh Token (24 hrs) stored inside the client header context, assuring secure API verification without backend sessions.

6.2 Dashboard Module

Maintains role-based UI views. The Freelancer Dashboard showcases application statistics, skills, portfolio projects, and bid history. The Company Dashboard tracks posted job openings, active RFPs, proposal summaries, and subscription plans.

6.3 Functional Modules

- **Jobs Module:** Creates postings, lists location, salary range, and job types. Handles PDF uploads.
- **RFP Bidding Module:** Enables companies to list budget contracts and categories. Freelancers attach bidding proposals.
- **Messaging Module:** Manages inbox lists and delivers asynchronous chats between clients.
- **Reviews & Ratings Module:** Computes average rating decimals out of 5 stars.

6.4 Administrative Module

Exposes administrative controls using Django Admin. Administrators override subscriptions, flag inappropriate reviews, ban fraudulent profiles, and inspect system-wide system configurations.

6.5 Reporting and Analytics Module

Displays real-time application trackers and charts. Aggregates values like active biddings, application pipelines, hiring metrics, and subscription transaction histories.

7. Functional Features

7.1 End User Features

- **Freelancer Portfolios:** Upload and list school degrees, companies worked for, and project URLs.
- **RFP Proposals:** Bid on company budget contracts, attaching documentation (PDF/DOCX/ZIP) up to 10MB.
- **Job Applications:** One-click application using default resume files.
- **Reviews System:** Write feedback and view average rating stars.

7.2 Administrative Features

- **User Moderation:** Inspect and suspend user accounts violating terms.
- **Plan Billing:** Add/edit posting credit packages (Basic, Standard, Premium).
- **Content Flagging:** Quarantine flagged review comments for manual review.

7.3 Security Features

- OTP validity window enforcement, JWT blacklisting, rate-limited request structures, and input validations.

8. RESTful API Specification

8.1 API Architecture

The system relies on a stateless REST API where all data payloads are serialized in standard JSON format. File transfers (resumes, logos) are handled as Multipart Form Data.

8.2 API Endpoints

Method	Endpoint	Auth	Description
POST	/api/token/	Public	User Login, returns JWT Access/Refresh tokens
POST	/api/token/refresh/	Public	Refresh expired JWT Access token
POST	/api/otp/send/	Public	Generates and emails 6-digit OTP code
POST	/api/otp/verify/	Public	Verifies OTP code validity
POST	/api/users/register/	Public	Register user account after OTP verification
GET/PUT	/api/me/	JWT	Fetch/Update current user profile info
GET/POST	/api/companies/	JWT	List/Create corporate company profiles
GET/POST	/api/jobs/	JWT	List job openings / Create a new posting
GET/POST	/api/applications/	JWT	Submit job application with resume file
GET/POST	/api/rfps/	JWT	List RFPs / Post a new Request for Proposal
GET/POST	/api/rfp-interests/	JWT	Submit proposal bidding document for RFP
GET/POST	/api/messages/	JWT	List chats / Send direct instant messages
GET	/api/notifications/	JWT	Retrieve user notifications

8.3 Request and Response Formats

Example Request Payload for OTP verification:

```
POST /api/otp/verify/
{
  "email": "user@domain.com",
  "otp": "482910"
}
```

Example Response (200 OK):

```
{
  "message": "OTP verified successfully",
  "verified": true
}
```

8.4 Error Handling and Status Codes

- **400 Bad Request:** Missing parameters or invalid email format.
- **401 Unauthorized:** Expired or missing JWT bearer token.
- **403 Forbidden:** Attempting actions without subscription credits.
- **404 Not Found:** Job opening or RFP resource does not exist.

9. User Interface Design

9.1 Home Page

Features a modern hero section containing search tags, navigation toggles, job category grids, and client statistics counters designed using glassmorphism components.

9.2 Authentication Screens

Dual-step layout: Inputting email displays the OTP input fields, and verification transitions to credentials registration form fields.

9.3 Dashboard Interface

Grid structures containing key performance card elements: active applications, unread message badges, credits remaining, and direct access controls.

9.4 Module Interfaces

RFP proposal forms display drag-and-drop file uploader zones (restricted to .pdf, .docx, .zip) complete with upload progress bars.

9.5 Administration Panel

Responsive tabular structures containing user grids, billing status buttons, review report toggles, and system analytics.

9.6 Responsive User Interface Design

Built using Chakra UI's responsive media breakpoint arrays (base, md, lg), adjusting columns, font sizes, and navigation menus automatically between phones, tablets, and desktop displays.

10. System Testing

10.1 Test Strategy

Testing follows an agile approach with Unit Testing for API serializers, Integration Testing for the OTP-registration flow, and System Testing for the job application process.

10.2 Core Test Cases

ID	Test Case Description	Input Data	Expected Output	Status
TC-01	OTP Generation and email delivery	email: test@xanatz.com	OTP sent via SMTP, returns 200	PASS
TC-02	User Registration with expired OTP	otp: 000000 (expired)	Returns 400 'Expired OTP'	PASS
TC-03	Job application without attachment	resume: empty	Rejects submission, asks for file	PASS
TC-04	RFP Bid file size validation	attached_file: 15MB .zip	Shows error: 'File exceeds 10MB'	PASS
TC-05	Direct Message send when authorized	recipient_id: 5, msg: 'Hi'	Saves message, triggers notification	PASS
TC-06	Double review submission check	rfp_interest_id: 2	API returns 400 'Already reviewed'	PASS

11. Security Implementation

11.1 Authentication Mechanism

Authentication uses JSON Web Tokens (JWT). The transient Access Token acts as a stateless session, while the Refresh Token is securely stored on the client side to request new keys, minimizing vulnerabilities to Cross-Site Scripting (XSS) attacks.

11.2 Authorization and Access Control

Enforced using DRF permission classes (`IsAuthenticated`, `IsAdminUser`). Access controls isolate company subscription actions from freelancer roles.

11.3 Password Encryption and Hashing

Uses Django's default PBKDF2 algorithm with a SHA-256 hash. Passwords are salted and iterated 600,000 times, ensuring defense against brute-force attacks.

11.4 CSRF Protection

Django's CSRF middleware generates unique validation tokens for all state-changing operations (POST, PUT, DELETE), preventing cross-origin request forgery.

11.5 Input Validation and Data Sanitization

All data payloads are serialized and sanitized using DRF Serializers. The Django ORM parameterizes all SQL operations, mitigating SQL Injection vulnerabilities.

12. Future Enhancements

- **Payment Gateway:** Integrate Stripe or Razorpay for subscription payment automation.
- **Video Interviewing:** Implement direct WebRTC-based video meetings between clients.
- **AI Recommender:** Use Machine Learning (NLP) to auto-match profiles with job descriptions.
- **Escrow Payments:** Establish smart-contract escrow releases upon milestone completions.

13. Conclusion

Xanatz successfully delivers a robust, secure, and modern freelance workspace. The platform demonstrates that Django's scalable model logic combined with React's modular UI architecture provides a highly cohesive, performant, and maintainable software product suitable for real-world operations.

14. References

1. Django Web Framework Documentation: <https://docs.djangoproject.com/>
2. Django REST Framework API Guides: <https://www.django-rest-framework.org/>
3. ReactJS Client Architecture: <https://react.dev/>

4. Chakra UI Web Styles & Layouts: <https://chakra-ui.com/>

15. Appendices

15.1 Database Scripts (SQL)

Below are the auto-generated DDL SQL scripts compiled from the current active Django database models, ready for deployment in MySQL.

```
CREATE TABLE `accounts_user` (
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `password` VARCHAR(128) NOT NULL,
  `last_login` DATETIME NULL,
  `is_superuser` TINYINT(1) NOT NULL DEFAULT False,
  `first_name` VARCHAR(150) NULL,
  `last_name` VARCHAR(150) NULL,
  `is_staff` TINYINT(1) NOT NULL DEFAULT False,
  `is_active` TINYINT(1) NOT NULL DEFAULT True,
  `date_joined` DATETIME NOT NULL,
  `email` VARCHAR(254) NOT NULL UNIQUE,
  `phone_number` VARCHAR(15) NULL,
  `accepted_privacy_policy` TINYINT(1) NOT NULL DEFAULT False
);

CREATE TABLE `accounts_profile` (
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `user_id` INT UNSIGNED NOT NULL,
  `public_id` CHAR(36) NOT NULL UNIQUE DEFAULT uuid.uuid4,
  `headline` VARCHAR(255) NULL,
  `about` TEXT NULL,
  `location` VARCHAR(255) NULL,
  `profile_picture` VARCHAR(100) NULL,
  `cover_image` VARCHAR(100) NULL,
  `website` VARCHAR(200) NULL,
  `skills` JSON NULL,
  `is_freelancer` TINYINT(1) NOT NULL DEFAULT False,
  `hourly_rate` DECIMAL(10,2) NULL,
  `freelancer_currency` VARCHAR(10) NOT NULL DEFAULT `USD`,
  `freelancer_availability` VARCHAR(20) NOT NULL DEFAULT `available`,
  CONSTRAINT `fk_accounts_profile_user_id` FOREIGN KEY (`user_id`) REFERENCES `accounts_user`
  (`id`) ON DELETE CASCADE
);

CREATE TABLE `accounts_portfolioprotect` (
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `profile_id` INT UNSIGNED NOT NULL,
  `title` VARCHAR(255) NOT NULL,
  `description` TEXT NULL,
  `image` VARCHAR(100) NULL,
  `project_url` VARCHAR(200) NULL,
  `technologies` JSON NULL,
  CONSTRAINT `fk_accounts_portfolioprotect_profile_id` FOREIGN KEY (`profile_id`) REFERENCES
  `accounts_profile` (`id`) ON DELETE CASCADE
);

CREATE TABLE `accounts_experience` (
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `profile_id` INT UNSIGNED NOT NULL,
  `company` VARCHAR(255) NOT NULL,
  `title` VARCHAR(255) NOT NULL,
  `location` VARCHAR(255) NULL,
  `start_date` DATE NOT NULL,
  `end_date` DATE NULL,
  `current` TINYINT(1) NOT NULL DEFAULT False,
  `description` TEXT NULL,
```

```
    `company_website` VARCHAR(200) NULL,  
    CONSTRAINT `fk_accounts_experience_profile_id` FOREIGN KEY (`profile_id`) REFERENCES  
    `accounts_profile` (`id`) ON DELETE CASCADE  
);
```

```
CREATE TABLE `accounts_education` (  
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    `profile_id` INT UNSIGNED NOT NULL,  
    `school` VARCHAR(255) NOT NULL,  
    `degree` VARCHAR(255) NULL,  
    `field_of_study` VARCHAR(255) NULL,  
    `start_date` DATE NOT NULL,  
    `end_date` DATE NULL,  
    `description` TEXT NULL,  
    CONSTRAINT `fk_accounts_education_profile_id` FOREIGN KEY (`profile_id`) REFERENCES  
    `accounts_profile` (`id`) ON DELETE CASCADE  
);
```

```
CREATE TABLE `accounts_otp` (  
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    `email` VARCHAR(254) NOT NULL,  
    `otp` VARCHAR(6) NOT NULL,  
    `created_at` DATETIME NOT NULL,  
    `is_verified` TINYINT(1) NOT NULL DEFAULT False  
);
```

```
CREATE TABLE `accounts_privacypolicy` (  
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    `content` TEXT NOT NULL,  
    `updated_at` DATETIME NOT NULL,  
    `created_at` DATETIME NOT NULL  
);
```

```
CREATE TABLE `accounts_companymember` (  
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    `user_id` INT UNSIGNED NOT NULL,  
    `company_id` INT UNSIGNED NOT NULL,  
    `access_role` VARCHAR(20) NOT NULL DEFAULT `user`,  
    `position` VARCHAR(255) NULL,  
    `joined_at` DATETIME NOT NULL,  
    CONSTRAINT `fk_accounts_companymember_user_id` FOREIGN KEY (`user_id`) REFERENCES  
    `accounts_user` (`id`) ON DELETE CASCADE,  
    CONSTRAINT `fk_accounts_companymember_company_id` FOREIGN KEY (`company_id`) REFERENCES  
    `accounts_company` (`id`) ON DELETE CASCADE  
);
```

```
CREATE TABLE `accounts_company` (  
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    `name` VARCHAR(255) NOT NULL,  
    `public_id` CHAR(36) NOT NULL UNIQUE DEFAULT uuid.uuid4,  
    `tagline` VARCHAR(255) NULL,  
    `description` TEXT NULL,  
    `logo` VARCHAR(100) NULL,  
    `website` VARCHAR(200) NULL,  
    `industry` VARCHAR(50) NULL,  
    `company_size` VARCHAR(20) NULL,  
    `location` VARCHAR(255) NULL,  
    `founded_year` INT UNSIGNED NULL,  
    `linkedin_url` VARCHAR(200) NULL,  
    `twitter_url` VARCHAR(200) NULL,  
    `is_active` TINYINT(1) NOT NULL DEFAULT True,  
    `creator_id` INT UNSIGNED NULL,  
    `created_at` DATETIME NOT NULL,
```

```

        `updated_at` DATETIME NOT NULL,
        CONSTRAINT `fk_accounts_company_creator_id` FOREIGN KEY (`creator_id`) REFERENCES
`accounts_user` (`id`) ON DELETE CASCADE
    );

CREATE TABLE `accounts_jobopening` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `company_id` INT UNSIGNED NOT NULL,
    `title` VARCHAR(255) NOT NULL,
    `description` TEXT NOT NULL,
    `requirements` TEXT NULL,
    `location` VARCHAR(255) NULL,
    `job_type` VARCHAR(50) NOT NULL DEFAULT `full_time`,
    `salary_range` VARCHAR(100) NULL,
    `is_active` TINYINT(1) NOT NULL DEFAULT True,
    `expires_at` DATETIME NULL,
    `created_at` DATETIME NOT NULL,
    `updated_at` DATETIME NOT NULL,
    CONSTRAINT `fk_accounts_jobopening_company_id` FOREIGN KEY (`company_id`) REFERENCES
`accounts_company` (`id`) ON DELETE CASCADE
    );

CREATE TABLE `accounts_jobapplication` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `job_opening_id` INT UNSIGNED NOT NULL,
    `applicant_id` INT UNSIGNED NOT NULL,
    `full_name` VARCHAR(255) NOT NULL,
    `email` VARCHAR(254) NOT NULL,
    `cover_letter` TEXT NULL,
    `resume` VARCHAR(100) NULL,
    `portfolio_url` VARCHAR(200) NULL,
    `key_skills` TEXT NULL,
    `status` VARCHAR(50) NOT NULL DEFAULT `applied`,
    `created_at` DATETIME NOT NULL,
    `updated_at` DATETIME NOT NULL,
    CONSTRAINT `fk_accounts_jobapplication_job_opening_id` FOREIGN KEY (`job_opening_id`)
REFERENCES `accounts_jobopening` (`id`) ON DELETE CASCADE,
    CONSTRAINT `fk_accounts_jobapplication_applicant_id` FOREIGN KEY (`applicant_id`) REFERENCES
`accounts_user` (`id`) ON DELETE CASCADE
    );

CREATE TABLE `accounts_rfp` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `company_id` INT UNSIGNED NOT NULL,
    `title` VARCHAR(255) NOT NULL,
    `description` TEXT NOT NULL,
    `budget` VARCHAR(100) NULL,
    `deadline` DATE NULL,
    `requirements` TEXT NULL,
    `category` VARCHAR(100) NULL,
    `sub_category` VARCHAR(100) NULL,
    `is_active` TINYINT(1) NOT NULL DEFAULT True,
    `created_at` DATETIME NOT NULL,
    `updated_at` DATETIME NOT NULL,
    CONSTRAINT `fk_accounts_rfp_company_id` FOREIGN KEY (`company_id`) REFERENCES
`accounts_company` (`id`) ON DELETE CASCADE
    );

CREATE TABLE `accounts_rfpinterest` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `rfp_id` INT UNSIGNED NOT NULL,
    `user_id` INT UNSIGNED NOT NULL,
    `company_name` VARCHAR(255) NOT NULL,
    `email` VARCHAR(254) NOT NULL,

```

```

        `phone_number` VARCHAR(15) NULL,
        `proposal_summary` TEXT NOT NULL,
        `attached_file` VARCHAR(100) NULL,
        `status` VARCHAR(50) NOT NULL DEFAULT `pending`,
        `is_reviewed` TINYINT(1) NOT NULL DEFAULT False,
        `created_at` DATETIME NOT NULL,
        CONSTRAINT `fk_accounts_rfpinterest_rfp_id` FOREIGN KEY (`rfp_id`) REFERENCES `accounts_rfp`
        (`id`) ON DELETE CASCADE,
        CONSTRAINT `fk_accounts_rfpinterest_user_id` FOREIGN KEY (`user_id`) REFERENCES
        `accounts_user` (`id`) ON DELETE CASCADE
    );

CREATE TABLE `accounts_jobpostplan` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `name` VARCHAR(50) NOT NULL UNIQUE,
    `display_name` VARCHAR(50) NOT NULL,
    `price` DECIMAL(10,2) NOT NULL DEFAULT 0,
    `max_jobs` INT UNSIGNED NOT NULL,
    `job_duration_days` INT UNSIGNED NOT NULL,
    `description` TEXT NULL,
    `features` JSON NULL,
    `is_active` TINYINT(1) NOT NULL DEFAULT True,
    `created_at` DATETIME NOT NULL
);

CREATE TABLE `accounts_companysubscription` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `company_id` INT UNSIGNED NOT NULL,
    `plan_id` INT UNSIGNED NOT NULL,
    `jobs_used` INT UNSIGNED NOT NULL DEFAULT 0,
    `activated_at` DATETIME NOT NULL,
    `is_active` TINYINT(1) NOT NULL DEFAULT True,
    CONSTRAINT `fk_accounts_companysubscription_company_id` FOREIGN KEY (`company_id`)
REFERENCES `accounts_company` (`id`) ON DELETE CASCADE,
    CONSTRAINT `fk_accounts_companysubscription_plan_id` FOREIGN KEY (`plan_id`) REFERENCES
    `accounts_jobpostplan` (`id`) ON DELETE CASCADE
);

CREATE TABLE `accounts_notification` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `recipient_id` INT UNSIGNED NOT NULL,
    `sender_id` INT UNSIGNED NULL,
    `message` TEXT NOT NULL,
    `is_read` TINYINT(1) NOT NULL DEFAULT False,
    `created_at` DATETIME NOT NULL,
    `target_url` VARCHAR(255) NULL,
    CONSTRAINT `fk_accounts_notification_recipient_id` FOREIGN KEY (`recipient_id`) REFERENCES
    `accounts_user` (`id`) ON DELETE CASCADE,
    CONSTRAINT `fk_accounts_notification_sender_id` FOREIGN KEY (`sender_id`) REFERENCES
    `accounts_user` (`id`) ON DELETE CASCADE
);

CREATE TABLE `accounts_message` (
    `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `sender_id` INT UNSIGNED NOT NULL,
    `recipient_id` INT UNSIGNED NOT NULL,
    `content` TEXT NOT NULL,
    `is_read` TINYINT(1) NOT NULL DEFAULT False,
    `created_at` DATETIME NOT NULL,
    CONSTRAINT `fk_accounts_message_sender_id` FOREIGN KEY (`sender_id`) REFERENCES
    `accounts_user` (`id`) ON DELETE CASCADE,
    CONSTRAINT `fk_accounts_message_recipient_id` FOREIGN KEY (`recipient_id`) REFERENCES
    `accounts_user` (`id`) ON DELETE CASCADE
);

```

```

CREATE TABLE `accounts_companyreview` (
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `reviewer_id` INT UNSIGNED NOT NULL,
  `company_id` INT UNSIGNED NULL,
  `rfp_interest_id` INT UNSIGNED NULL,
  `company_name` VARCHAR(255) NOT NULL,
  `rating` INT UNSIGNED NOT NULL,
  `review_text` TEXT NOT NULL,
  `is_flagged` TINYINT(1) NOT NULL DEFAULT False,
  `created_at` DATETIME NOT NULL,
  CONSTRAINT `fk_accounts_companyreview_reviewer_id` FOREIGN KEY (`reviewer_id`) REFERENCES
`accounts_user` (`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_accounts_companyreview_company_id` FOREIGN KEY (`company_id`) REFERENCES
`accounts_company` (`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_accounts_companyreview_rfp_interest_id` FOREIGN KEY (`rfp_interest_id`)
REFERENCES `accounts_rfpinterest` (`id`) ON DELETE CASCADE
);

CREATE TABLE `accounts_freelancerreview` (
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `reviewer_id` INT UNSIGNED NOT NULL,
  `freelancer_id` INT UNSIGNED NOT NULL,
  `rfp_interest_id` INT UNSIGNED NULL,
  `rating` INT UNSIGNED NOT NULL,
  `review_text` TEXT NOT NULL,
  `is_flagged` TINYINT(1) NOT NULL DEFAULT False,
  `created_at` DATETIME NOT NULL,
  CONSTRAINT `fk_accounts_freelancerreview_reviewer_id` FOREIGN KEY (`reviewer_id`) REFERENCES
`accounts_user` (`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_accounts_freelancerreview_freelancer_id` FOREIGN KEY (`freelancer_id`)
REFERENCES `accounts_user` (`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_accounts_freelancerreview_rfp_interest_id` FOREIGN KEY (`rfp_interest_id`)
REFERENCES `accounts_rfpinterest` (`id`) ON DELETE CASCADE
);

```

15.2 API Documentation and FAQ

Q: What are the file formats allowed for resume and proposal uploads?

A: Resumes support PDF, DOC, and DOCX up to 5MB. Proposals support PDF, DOC, DOCX, ZIP, and RAR up to 10MB.

Q: How are OTP codes secured?

A: OTP codes are cryptographically generated, associated with the email address, and expire in 10 minutes.